

The aim of this script consists of designing a Sensor Network in charge to estimate the humidity level in different rooms of a building. All the sensors are defined by a common dynamic of discrete-time integrators described as follows:

$$x_{i,k+1} = x_{i,k} + u_{i,k}$$

With:

$x_{i,k} \in R :=$ state of the sensor (it contains the current value of humidity)

$u_{i,k} \in R :=$ input signal for the sensor

In order to obtain a good estimation of humidity, it is required to solve an Average Consensus Problem among the agents of the network. This means that over time, the sensors should collectively converge to a common estimate of the humidity level in the room represented by the average of the initial conditions.

Thus, the input associated to the i-th agent in the discrete consensus framework will be:

$$u_{i,k} = -\beta * \sum_{j=1, j \neq i}^n a_{ij} * (x_{i,k} - x_{j,k})$$

That it is to say, a summation of displacements between the state of the i-th sensor with respect to the state of its predecessors (nodes which convey information versus it). At the end, the final result is scaled by a scalar factor β .

Introducing the following vectors:

$$x_k = \begin{bmatrix} x_{1,k} \\ x_{2,k} \\ \vdots \\ x_{n,k} \end{bmatrix} \in R^n, \quad u_k = \begin{bmatrix} u_{1,k} \\ u_{2,k} \\ \vdots \\ u_{n,k} \end{bmatrix} \in R^n$$

We can define a global version for the evolution of the network as follows:

$$x_{k+1} = I * x_k - \beta * L * x_k = (I - \beta * L) * x_k$$

Where $L = D - A$ represents the Laplacian matrix.

Identifying $(I - \beta * L) = P$ (Perron Matrix), we obtain the final statement for the evolution of the discrete integrators under the action of the Consensus Protocol:

$$x_{k+1} = P * x_k$$

We assume that P is a row-stochastic matrix, that is to say the summation of the elements of each row is equal to one:

$$P * 1_n = 1_n$$

Thus, P has a unitary eigenvector associated to the eigenvalue equal to 1.

Now, the problem consists of ensuring the Network reaches a consensus whatever are the initial conditions. This is guaranteed by the following theorem:

THEOREM:

The system reaches a consensus if and only if the matrix P is primitive.

A square matrix P is said to be primitive if:

$$\exists k > 0 \text{ such that } P^k > 0$$

Hence, we want to ensure the row-stochasticness and primitiveness of the matrix P to be sure the system reaches a consensus. This two conditions are guaranteed by respecting the following fundamental theorem:

THEOREM:

A square matrix P is primitive and row-stochastic if:

1. the di-graph $\vec{G}$ is strongly connected.
2. It holds $0 < \beta < \frac{1}{\Delta}$, where Δ represents the maximum in-degree of the network.

A directed graph $\vec{G}(V, E)$ is strongly connected if for all pairs of nodes $i, j \in V$ there exists a path between them.

However, we need to ensure that the network solves an average consensus problem instead of a simple consensus problem. In general, the consensus value is equal to:

$$\bar{x} = \frac{w^T}{w^T * 1_n} * x_0 = \hat{w}^T * x_0$$

Where w^T is the left-eigenvector associated to the zero eigenvalue of the laplacian matrix.

If the graph is weight-balanced (in-degree equal to out-degree for each node) it holds:

$$w^T = 1_n$$

Hence, the consensus value becomes:

$$\bar{x} = \frac{1_n^T}{1_n^T * 1_n} * x_0 = \frac{1}{n} * 1_n^T * x_0 = \frac{1}{n} * \sum_{i=1}^n x_{i,0}$$

So, the consensus value is represented by the average of the initial conditions (Average Consensus Problem).

Once we have identified the conditions on the topology we can introduce further information about our case study. It's known that each device is affected by a noise during a measurement. The origin of the noise can be related to external noise (condition of the environment) or to internal noise (loss of accuracy of the device). In practice, this means that after a measurement each sensor could provide different values for the humidity despite they are located in the same room. For our purposes the noise has been modeled as a Normally

Gaussian distributed noise of zero mean and variance of 1. Over time, the sensors provide new measurements to store. As a consequence, the network should update its state and get ready to start a new consensus protocol on the new initial conditions.

Now we have all the information to perform our simulation.

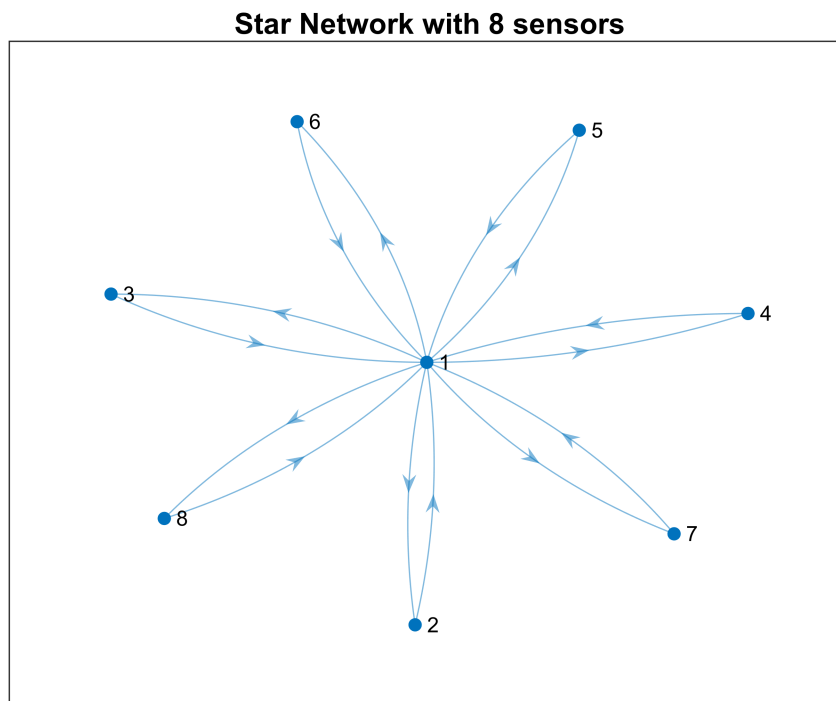
First, a topology for the network must be designed. In order to guarantee robustness, a star topology has been chosen. Star networks offer a good safety level because if some nodes go down the strongly connectiveness property continues to be verified (exception for a failure in the central node). Other topologies, such as ring graph or path graph, have bad response to nodes failure.

```
clear;close all;
```

STAR NETWORK

Let's create the topology:

```
n=8; %number of nodes
A=zeros(n,n); %adjacency matrix
A(1,2:end)=1;
for i=2:n
    A(i,1)=1;
end
g=digraph(A);
plot(g)
title("Star Network with "+n+" sensors");
```



Define the variables of interest for the application of the Consensus Protocol in the case of having a network composed of discrete-time integrators:

```
D=zeros(n,n); %degree matrix
for i=1:n
    D(i,i)=indegree(g,i);
end
L=D-A;%laplacian matrix
indegree_max=max(indegree(g));%maximum indegree in the network (N-1 for this topology)
beta=(1/indegree_max)-((1/indegree_max)/10); % 0<beta<(1/indegree_max)
P=eye(n)-beta*L;%Perron matrix
```

Let's define some variables of interest:

```
real_value=65; %real value of humidity represented by percentage
x0=(real_value+randn(1,n))'; %measurements affected by a Normally Distributed Gaussian noise
Tsim=6*n;%simulation time
average=sum(x0)/n;%current average
x=x0; %state variable
```

Let's derive an empirical result on the Sampling Time T_s : we want to guarantee about 50 samples in the transient T . The duration of T can be approximated as follows:

$$T = 4 * \tau$$

where τ is the time-constant of the system and it is equal to:

$$\tau = \frac{1}{|p_{\text{dominant}}|}$$

With p_{dominant} representing the dominant eigenvalue (closest to one) of the dynamic matrix P .

This means that the convergence time of the network can be approximated by considering T .

In order to set the sampling time we can use the following rule of thumb:

$$T_s = \frac{T}{\text{\#samples in } T}$$

```
lambdas=sort(abs(eig(P))); % eigenvalues of P ordered in ascending way
tau=1/lambdas(end-1); % time constant of the system
Ts=(4*tau)/50; %sampling time
```

Let's implement the Consensus Protocol: we want to simulate the evolution of the Network in the condition of performing n measurements (equal to the number of nodes) every $50 \cdot T_s$ seconds.

```
k=1;% index for the average values
j=1; % index for the time-out of the measurements
N=1; %number of measurements
```

```

for i=Ts:Ts:Tsim
    if(j==50 && N<n) % Time-out (new incoming measurement)
        x0=(real_value+randn(1,n))';
        x(:,k+1)=x0;
        average(k+1)=sum(x0)/n;
        k=k+1;
        j=1;N=N+1;
        continue
    end
    x(:,k+1)=P*x(:,k);
    average(k+1)=sum(x0)/n;
    k=k+1;j=j+1;
end

```

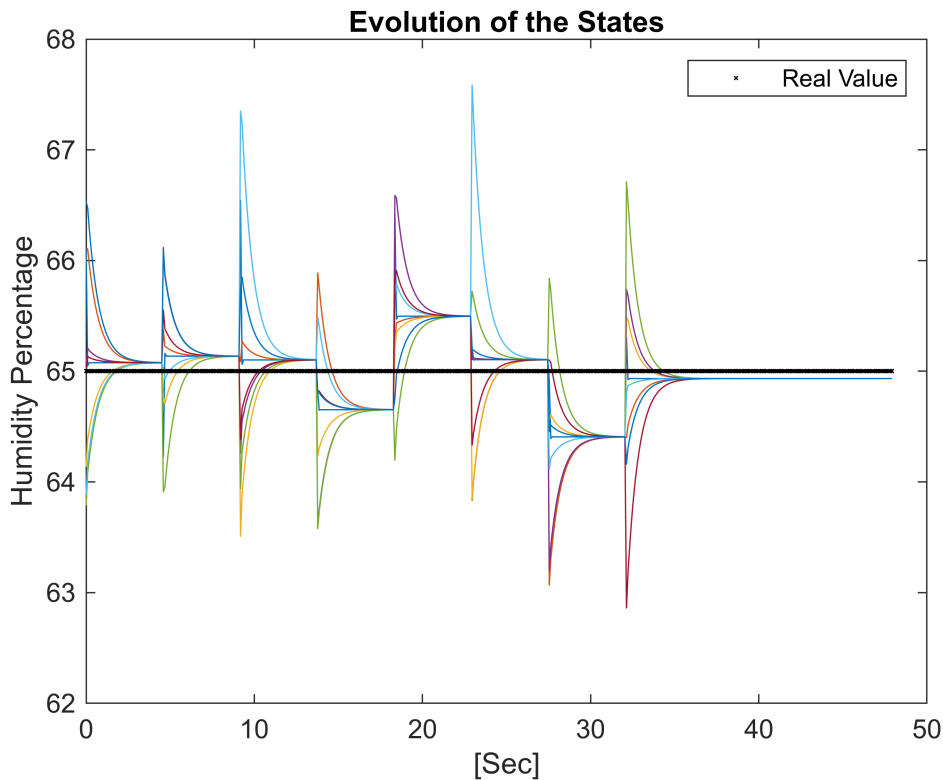
Plots of the results:

```

figure
plot(0:Ts:Tsim,x)
title("Evolution of the States");xlabel("[Sec]");ylabel("Humidity Percentage");
hold on;
real_value_plot=plot(0:Ts:Tsim,real_value,'kx','MarkerSize',2);
legend(real_value_plot,"Real Value");

```

Warning: Ignoring extra legend entries.



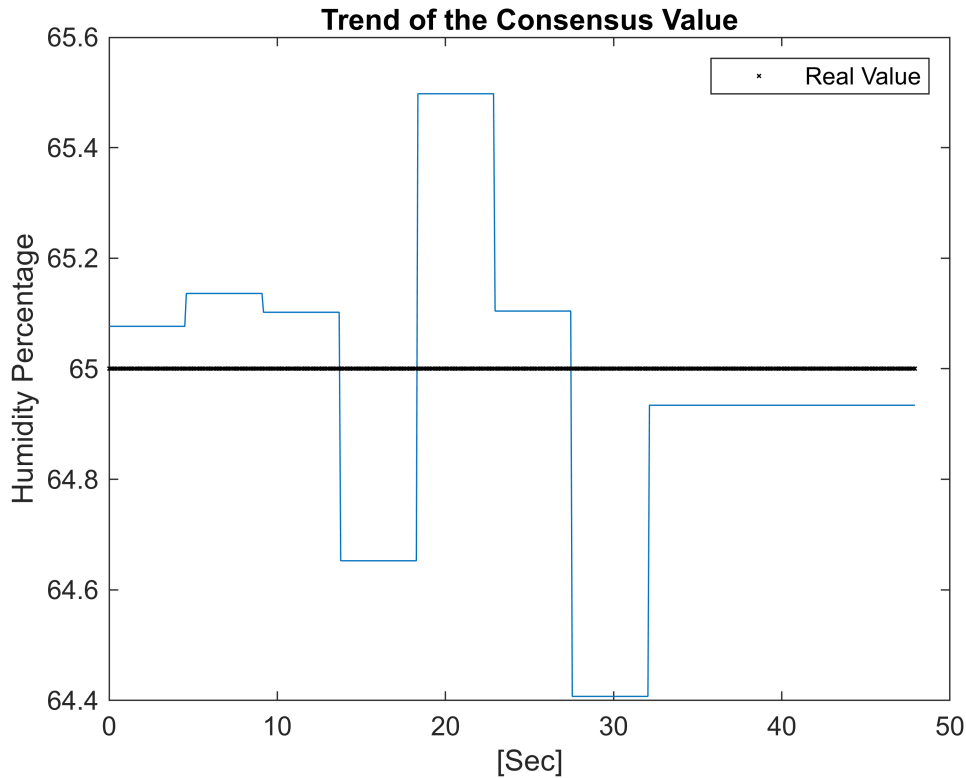
```

figure;
plot(0:Ts:Tsim,average);
title("Trend of the Consensus Value");xlabel("[Sec]");ylabel("Humidity Percentage");
hold on;

```

```
real_value_plot=plot(0:Ts:Tsim,real_value,'kx','MarkerSize',2);
legend(real_value_plot,"Real Value");
```

Warning: Ignoring extra legend entries.



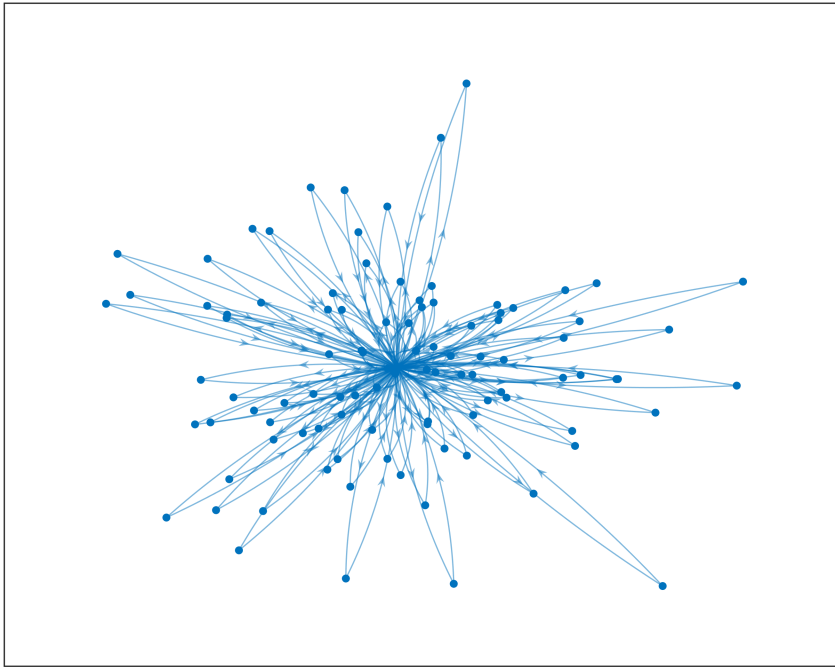
Now the problem consists of improving the accuracy of the result. When we operate with sensor Network most of the time we can act in two different ways: it's possible to add new edges or it's possible to use more sensors. The first choice doesn't affect the accuracy of the result because the increasement of the number of edges has effect only on the eigenvalues of L and, thus, on the algebraic connectivity λ_2 (second smallest eigenvalue of matrix L). Therefore, the new set up affects the convergence time instead of the accuracy, in fact increasing the number of edges we obtain a bigger algebraic connectivity (an indicator of the speed of convergence). So, it implies a reduction of the convergence time.

In order to improve the accuracy we can follow the second strategy, that is to say to increase the number of nodes.

Let's create the new network:

```
n_new=150; %number of nodes
A_new=zeros(n_new,n_new); %adjacency matrix
A_new(1,2:end)=1;
for i=2:n_new
    A_new(i,1)=1;
end
g_new=digraph(A_new);
figure
plot(g_new)
title("Star Network with "+n_new+" sensors");
```

Star Network with 150 sensors



And implement the protocol:

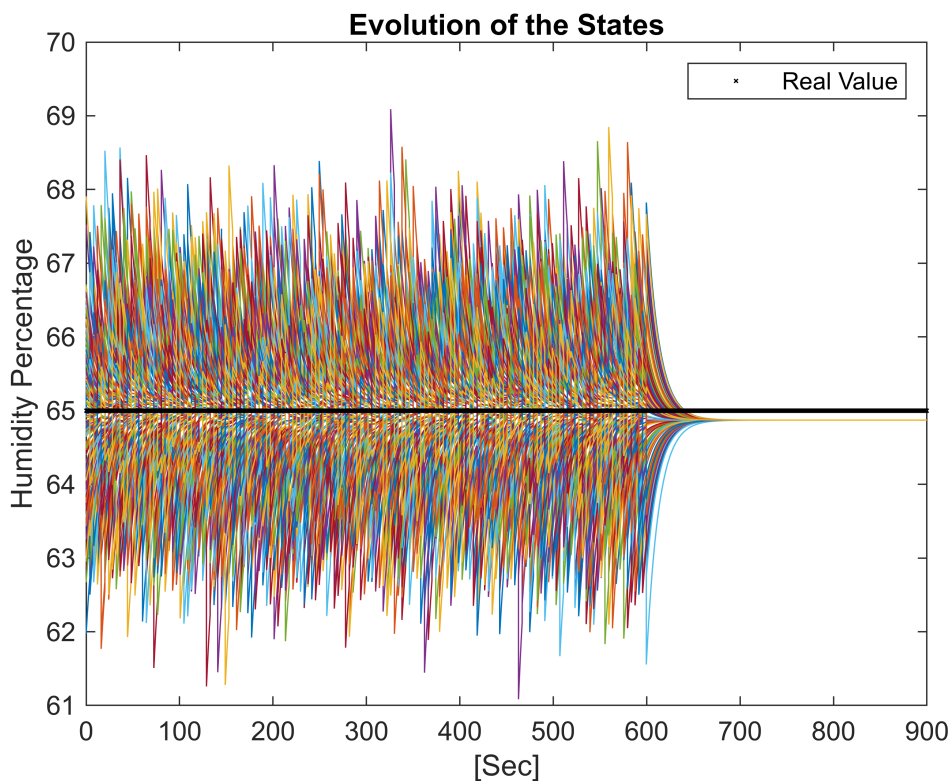
```
D_new=zeros(n_new,n_new); %degree matrix
for i=1:n_new
    D_new(i,i)=indegree(g_new,i);
end
L_new=D_new-A_new;%laplacian matrix
indegree_max_new=max(indegree(g_new));%maximum indegree in the network (N-1 for
this topology)
beta_new=(1/indegree_max_new)-((1/indegree_max_new)/10); % 0<beta<(1/indegree_max)
P_new=eye(n_new)-beta_new*L_new;%Perron matrix
x0_new=[x0',real_value+randn(1,n_new-length(x0))]' ; %measurements affected by a
Normally Dstributed Gaussian noise
Tsim_new=6*n_new;%simulation time
average_new=sum(x0_new)/n_new;%current average
x_new=x0_new; %state variable
lambdas_new=sort(abs(eig(P_new))); % eigenvalues of P ordered in ascending way
tau_new=1/lambdas_new(end-1); % time constant of the system
Ts_new=(4*tau_new)/50; %sampling time
k=1;% index for the average values
j=1; % index for the time-out of the measurements
N=1; %number of measurements
for i=Ts_new:Ts_new:Tsim_new
    if(j==50 && N<n_new) % Time-out (new incoming measurement)
        x0_new=(real_value+randn(1,n_new))';
        x_new(:,k+1)=x0_new;
        average_new(k+1)=sum(x0_new)/n_new;
```

```

        k=k+1;
        j=1;N=N+1;
        continue
    end
    x_new(:,k+1)=P_new*x_new(:,k);
    average_new(k+1)=sum(x0_new)/n_new;
    k=k+1;j=j+1;
end
figure
plot(0:Ts_new:Tsim_new,x_new)
title("Evolution of the States");xlabel("[Sec]");ylabel("Humidity Percentage");
hold on;
real_value_plot=plot(0:Ts_new:Tsim_new,real_value,'kx','MarkerSize',2);
legend(real_value_plot,"Real Value");

```

Warning: Ignoring extra legend entries.

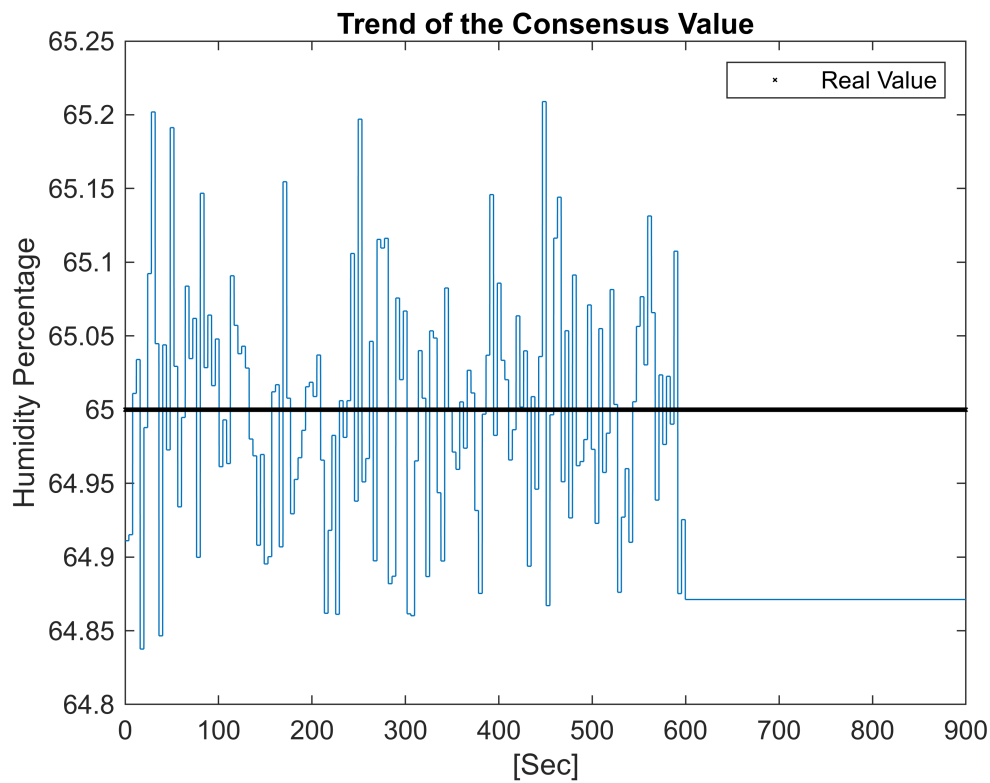


```

figure;
plot(0:Ts_new:Tsim_new,average_new);
title("Trend of the Consensus Value");xlabel("[Sec]");ylabel("Humidity Percentage");
hold on;
real_value_plot=plot(0:Ts_new:Tsim_new,real_value,'kx','MarkerSize',2);
legend(real_value_plot,"Real Value");

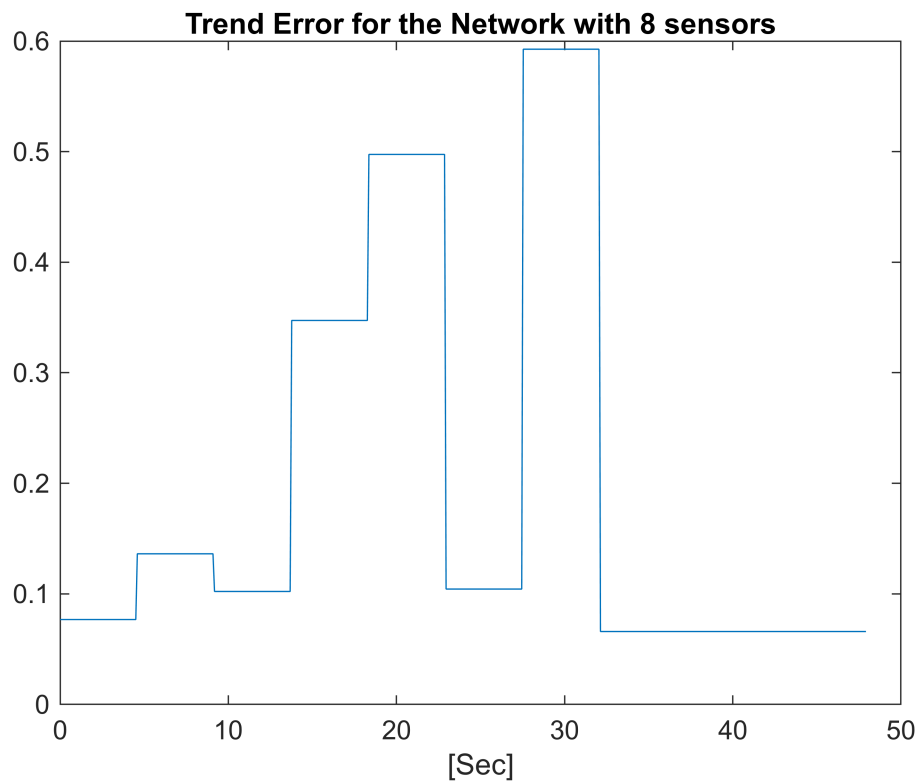
```

Warning: Ignoring extra legend entries.

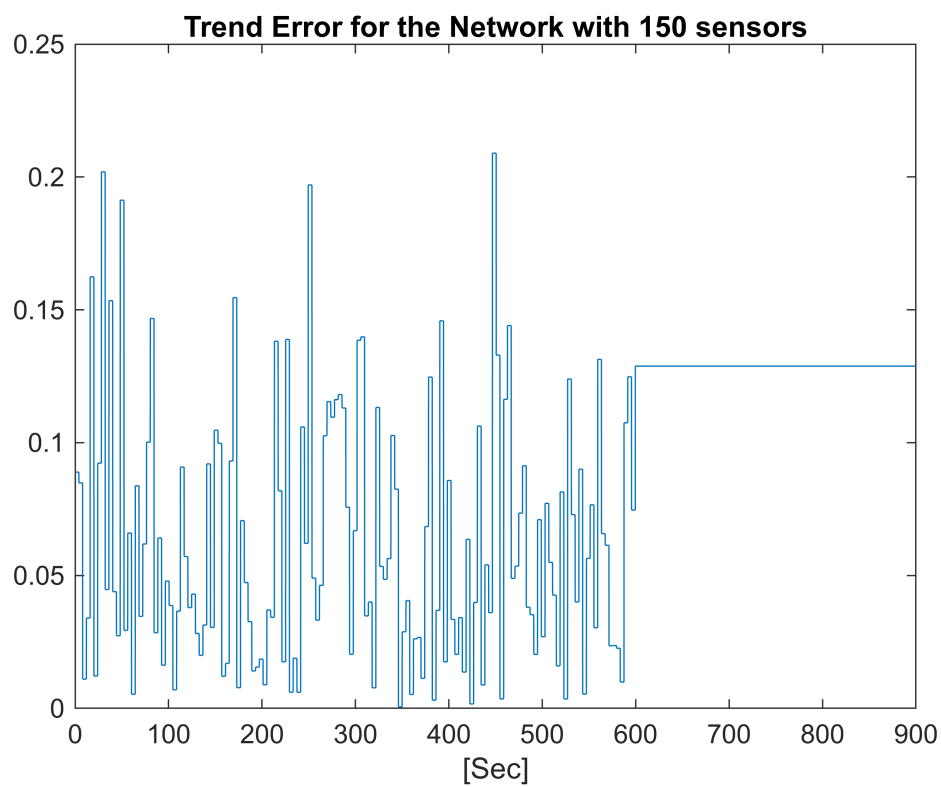


The simulation has considered an initial state equal to the previous measurements plus the measurements related to the new sensors. To appreciate the better behaviour of the network with more sensors in terms of accuracy, let's plot the trend error of the two networks considered:

```
e1=abs(average-real_value);
e2=abs(average_new-real_value);
figure
plot(0:Ts:Tsim,e1);title("Trend Error for the Network with "+n+"
sensors");xlabel("[Sec]");
```



```
figure
plot(0:Ts_new:Tsim_new,e2);title("Trend Error for the Network with "+n_new+"
sensors");xlabel("[Sec]");
```



We can note that the network with 150 sensors is more accurated.